C++ Class and Object (Easy Explanation)

🔹 What is a Class?

A **class** is a **blueprint** or **template** for creating objects.
It defines **data (variables)** and **behavior (functions)** together.

👉 Think like this:

- **Class** = Design of a house
- **Object** = Actual house built from the design

🔹 What is an Object?

An **object** is a **real instance** of a class.
Using an object, we can **access variables and functions** of the class.

🔹 Simple Example in C++

```cpp
#include <iostream>
using namespace std;

// Class declaration
class Student {
public:
    int id;
    string name;

    void showInfo() {
        cout << "ID: " << id << endl;
        cout << "Name: " << name << endl;
    }
};

int main() {
    // Object creation
    Student s1;

    s1.id = 101;
    s1.name = "Atiar";

    s1.showInfo();
```

```
    return 0;
}
```

🔹 Output

```
ID: 101
Name: Atiar
```

🔹 Parts of a Class

Part	Description
`class`	Keyword to create class
Data Members	Variables inside class
Member Functions	Functions inside class
`public`	Accessible from outside

🔹 Access Specifiers

```
class Example {
private:
    int a;   // Only inside class

protected:
    int b;   // Derived class access

public:
    int c;   // Anywhere access
};
```

🔹 Constructor (Special Function)

A **constructor** is automatically called when an object is created.

```
class Car {
public:
    Car() {
```

```
        cout << "Car object created!" << endl;
    }
};

int main() {
    Car c1;
}
```

🔹 Object Using Constructor

```
class Student {
public:
    int id;
    string name;

    Student(int i, string n) {
        id = i;
        name = n;
    }

    void display() {
        cout << id << " " << name << endl;
    }
};

int main() {
    Student s1(1, "Rahman");
    s1.display();
}
```

🔹 Why Use Class & Object?

✔ Code reusability
✔ Data security
✔ Real-world modeling
✔ Better structure (OOP)

🔹 Inheritance in OOP (C++)

✅ What is Inheritance?

Inheritance means **one class (child/derived)** acquires the **properties and behaviors** of another class (**parent/base**).

👉 Purpose:

- Code reuse
- Easy maintenance
- Real-world relationship (IS-A relationship)

Example:
`Teacher IS A Person`
`Car IS A Vehicle`

🔹 Basic Syntax

```
class Parent {
    // members
};

class Child : access_specifier Parent {
    // members
};
```

🔹 Simple Example

```
#include <iostream>
using namespace std;

// Base class
class Person {
public:
    string name;

    void speak() {
        cout << "Person can speak" << endl;
    }
};

// Derived class
class Student : public Person {
public:
    void study() {
        cout << name << " is studying" << endl;
```

```
    }
};

int main() {
    Student s;
    s.name = "Atiar";
    s.speak();   // inherited
    s.study();

    return 0;
}
```

🔹 Output

```
Person can speak
Atiar is studying
```

🔹 Types of Inheritance in C++

1️⃣ Single Inheritance

One child, one parent

```
class A {
public:
    void show() {
        cout << "Class A" << endl;
    }
};

class B : public A {};
```

2️⃣ Multilevel Inheritance

Inheritance chain

```
class A {
public:
    void a() { cout << "A" << endl; }
};

class B : public A {
```

```
public:
    void b() { cout << "B" << endl; }
};

class C : public B {
public:
    void c() { cout << "C" << endl; }
};
```

3️⃣ Multiple Inheritance

One class inherits from **multiple parents**

```
class A {
public:
    void a() { cout << "A" << endl; }
};

class B {
public:
    void b() { cout << "B" << endl; }
};

class C : public A, public B {};
```

⚠️ Problem: **Diamond Problem**

4️⃣ Hierarchical Inheritance

One parent, multiple children

```
class Animal {
public:
    void eat() {
        cout << "Eating" << endl;
    }
};

class Dog : public Animal {};
class Cat : public Animal {};
```

5️⃣ Hybrid Inheritance

Combination of inheritance types

🔹 Access Specifiers in Inheritance

Parent Member	Public Inheritance	Protected	Private
public	public	protected	private
protected	protected	protected	private
private	❌ Not inherited	❌	❌

🔹 Diamond Problem & Virtual Inheritance

❌ Problem

```
class A {
public:
    int x;
};

class B : public A {};
class C : public A {};
class D : public B, public C {};
```

👉 `D` gets **two copies** of `A`.

✅ Solution: `virtual`

```
class A {
public:
    int x;
};

class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {};
```

Now only **one copy** of `A`.

🔹 Function Overriding

```
class Parent {
public:
    void show() {
        cout << "Parent show" << endl;
    }
};

class Child : public Parent {
public:
    void show() {
        cout << "Child show" << endl;
    }
};

int main() {
    Child c;
    c.show();
}
```

🔹 `base class` Function Call

```
Parent::show();
```

🔹 Real-World Example

```
class Account {
public:
    double balance;

    void deposit(double amount) {
        balance += amount;
    }
};

class SavingsAccount : public Account {
public:
    void addInterest() {
        balance += balance * 0.05;
```

```
    }
};
```

🔹 Advantages of Inheritance

✔ Code reusability
✔ Extensibility
✔ Cleaner code
✔ Supports polymorphism

🔹 Interview Tips

- Prefer **composition over inheritance**
- Use **virtual inheritance** carefully
- Use `protected` for inheritance
- Avoid deep inheritance chains

```
#include<bits/stdc++.h>
using namespace std;

class Student{
    public:
    char name[100];
    int roll;
    double gpa;
};

int main(){

    Student a;
    char temp[100]="atiar";
    strcpy(a.name,temp);
    a.roll = 12;
    a.gpa=3.4;

    cout<<a.name<<" "<<a.roll<<" "<<a.gpa<<endl;
```

```
    return 0;
}
```

🔹 Code Explanation

```
#include<bits/stdc++.h>
```

✔ Includes **all standard C++ libraries** at once
✔ Common in competitive programming
(Internally includes `<iostream>`, `<cstring>`, etc.)

```
using namespace std;
```

✔ Allows you to use `cout`, `endl`, `strcpy` without writing `std::`

🔹 Class Definition

```
class Student{
public:
    char name[100];
    int roll;
    double gpa;
};
```

Explanation:

- `class Student` → User-defined data type
- `public:` → Members accessible outside the class
- `char name[100];` → Stores student name as C-style string
- `int roll;` → Roll number
- `double gpa;` → GPA value

👉 This class only **stores data**, no functions

🔹 main() Function

```
int main(){
```

Program execution starts here

```
Student a;
```

✔ Creates an **object** `a` of class `Student`

```
char temp[100] = "atiar";
```

✔ Temporary character array to store name
✔ `"atiar"` is a **string literal**

```
strcpy(a.name, temp);
```

✔ Copies string from `temp` to `a.name`
✔ Required because **char arrays cannot be assigned directly**

❌ Invalid:

```
a.name = temp;  // not allowed
```

```
a.roll = 12;
a.gpa = 3.4;
```

✔ Assign values to object data members using **dot operator (`.`)**

```
cout << a.name << " " << a.roll << " " << a.gpa << endl;
```

✔ Prints student information

Output:

```
atiar 12 3.4
```

```
return 0;
}
```

✔ Ends program successfully

🔹 Important Concepts Used

Concept	Explanation
Class	User-defined data type
Object	Instance of class
Dot Operator	Access class members
C-string	Character array
strcpy()	Copy string to char array

🔹 Better Modern C++ Version (Recommended)

```
class Student {
public:
    string name;
    int roll;
    double gpa;
};

int main() {
    Student a;
    a.name = "atiar";
    a.roll = 12;
    a.gpa = 3.4;

    cout << a.name << " " << a.roll << " " << a.gpa << endl;
}
```

✔ Safer
✔ No `strcpy()`
✔ Cleaner code

🔹 Interview Tip

❓ Why `strcpy()` needed?
✅ Because **char arrays cannot be assigned directly**

Here are **two correct ways** to take **input from user** for your `Student` class.
I'll explain both **C-style (char array)** and **modern C++ (string)** 👇

✅ 1️⃣ Input Using `char name[ ]` (Your Current Style)

```
#include <bits/stdc++.h>
using namespace std;

class Student {
public:
    char name[100];
    int roll;
    double gpa;
};

int main() {
    Student a;

    cout << "Enter name: ";
    cin.getline(a.name, 100);   // for full name

    cout << "Enter roll: ";
    cin >> a.roll;

    cout << "Enter GPA: ";
    cin >> a.gpa;

    cout << "\nStudent Info:\n";
    cout << a.name << " " << a.roll << " " << a.gpa << endl;

    return 0;
}
```

🔹 Why `cin.getline()`?

- `cin >> a.name` ❌ stops at space
- `cin.getline()` ✅ reads full name with spaces

⚠️ Common Mistake

If you use `cin >> roll` **before** `getline`, buffer issue occurs.

Fix:

```
cin.ignore();
cin.getline(a.name, 100);
```

✅ 2️⃣ Recommended: Input Using `string` (Modern C++)

```
#include <bits/stdc++.h>
using namespace std;

class Student {
public:
    string name;
    int roll;
    double gpa;
};

int main() {
    Student a;

    cout << "Enter name: ";
    getline(cin, a.name);

    cout << "Enter roll: ";
    cin >> a.roll;

    cout << "Enter GPA: ";
    cin >> a.gpa;

    cout << "\nStudent Info:\n";
    cout << a.name << " " << a.roll << " " << a.gpa << endl;

    return 0;
}
```

✔ No buffer issue
✔ No size limit
✔ Safer & cleaner

🔹 Sample Input

```
Enter name: Atiar Rahman
Enter roll: 12
Enter GPA: 3.4
```

🔹 Output

```
Student Info:
Atiar Rahman 12 3.4
```

🔹 Interview Tip

❓ Difference between `cin >>` and `getline()`
✅ `cin >>` → stops at space
✅ `getline()` → reads full line

🔹 `this` Keyword & Arrow (`->`) Operator in C++

I'll explain **both clearly**, with **simple examples** and **when/why to use them** 👇

🔹 1️⃣ `this` Keyword

✅ What is `this`?

`this` is a **pointer** that holds the **address of the current object**.

👉 It is automatically available **inside non-static member functions**.

🔹 Why Use `this`?

✔ To differentiate data members and parameters

✔ To return current object

✔ To pass current object as argument

🔹 Example 1: Name Conflict (Most Common)

```
class Student {
public:
    int roll;

    void setRoll(int roll) {
        this->roll = roll;  // object variable = parameter
    }
};

int main() {
    Student s;
    s.setRoll(12);
    cout << s.roll;
}
```

🔹 Without `this` ❌

```
roll = roll;   // useless (parameter assigned to itself)
```

🔹 Example 2: Return Current Object

```
class Test {
public:
    int x;

    Test set(int x) {
        this->x = x;
        return *this;
    }
};
```

🔹 Example 3: Pass Current Object

```
class Sample {
public:
    void show() {
        cout << "Hello" << endl;
    }
```

```
    void call(Sample *obj) {
        obj->show();
    }

    void test() {
        call(this);
    }
};
```

🔹 2️⃣ Arrow (`->`) Operator

✅ What is `->`?

Used to **access class members using a pointer object**.

🔹 Dot (`.`) vs Arrow (`->`)

Situation	Operator
Object	`.`
Pointer to object	`->`

🔹 Example: Dot Operator

```
Student s;
s.roll = 10;
```

🔹 Example: Arrow Operator

```
Student *p = &s;
p->roll = 20;
```

✔ Same as:

```
(*p).roll = 20;
```

🔹 `this` with Arrow Operator

Because `this` is a **pointer**, we use `->`

```
this->roll = roll;
```

🔹 Combined Example (`this` + `->`)

```
class Student {
public:
    string name;
    int roll;

    Student(string name, int roll) {
        this->name = name;
        this->roll = roll;
    }

    void display() {
        cout << name << " " << roll << endl;
    }
};
```

🔹 Real-World Analogy

- `this` = **"current object"**
- `->` = **"go to that object's member"**

🔹 Interview Questions

Q: Is `this` available in static functions?
❌ No

Q: Is `this` a pointer or reference?
✅ Pointer

Q: Why use `this->` instead of direct name?
✅ To avoid confusion and improve clarity

🔹 Quick Summary

Concept	Meaning
`this`	Pointer to current object
`->`	Access member via pointer
`.`	Access member via object

🔹 Returning an Object from a Function in C++

In C++, a function **can return an object of a class** just like returning `int` or `double`.

🔹 Why Return Objects?

✔ To create **new objects**
✔ To support **method chaining**
✔ To write clean, OOP-style code

✅ 1️⃣ Return Object by Value (Most Common)

```
#include <bits/stdc++.h>
using namespace std;

class Student {
public:
    string name;
    int roll;

    Student(string n, int r) {
        name = n;
        roll = r;
    }

    Student updateRoll(int r) {
        roll = r;
        return *this;   // returning object
    }

    void show() {
```

```
        cout << name << " " << roll << endl;
    }
};

int main() {
    Student s("Atiar", 10);

    Student s2 = s.updateRoll(20);
    s2.show();

    return 0;
}
```

🔹 Output

```
Atiar 20
```

🔹 Explanation

```
return *this;
```

- `this` → pointer to current object
- `*this` → current object itself

✅ 2️⃣ Method Chaining (Very Important)

```
class Box {
public:
    int length;

    Box set(int l) {
        length = l;
        return *this;
    }
};

int main() {
    Box b;
    b.set(5).set(10).set(20);
}
```

✔ Each function returns an object
✔ Next call happens on returned object

✅ 3️⃣ Return Object Created Inside Function

```
class Student {
public:
    string name;
    int roll;

    Student(string n, int r) {
        name = n;
        roll = r;
    }
};

Student createStudent() {
    Student s("Rahman", 12);
    return s;
}

int main() {
    Student s1 = createStudent();
    cout << s1.name << " " << s1.roll << endl;
}
```

✔ Safe
✔ Uses Return Value Optimization (RVO)

⚠️ 4️⃣ Return Object by Reference (Be Careful)

```
Student& func() {
    Student s("A", 1);
    return s;  // ❌ dangerous
}
```

❌ Local object destroyed after function ends

✅ Correct Way to Return Reference

```
Student& getStudent(Student &s) {
    return s;
}
```

🔹 5️⃣ Return Object Pointer (Optional)

```
Student* create() {
    return new Student("Atiar", 12);
}
```

⚠️ Must use `delete` later to avoid memory leak

🔹 Interview Summary

Return Type	Safe	Use Case
By Value	✅	Most common
By Reference	⚠️	Existing object
By Pointer	⚠️	Dynamic memory

🔹 Interview Q&A

Q: What does `return *this` mean?
✅ Returns current object

Q: Is returning object expensive?
✅ No (RVO + move semantics)

🔹 Dynamic Object Creation in C++ (Easy Explanation)

In C++, **dynamic object creation** means creating an object **at runtime** using **heap memory**.

👉 Keywords used:

- `new` → create object

- `delete` → destroy object

🔹 Why Dynamic Objects?

✔ Object lifetime controlled manually
✔ Used when size/number not known at compile time
✔ Required for polymorphism (base pointer → derived object)

✅ 1️⃣ Create Dynamic Object (Single Object)

```
#include <bits/stdc++.h>
using namespace std;

class Student {
public:
    string name;
    int roll;

    Student(string n, int r) {
        name = n;
        roll = r;
        cout << "Constructor called\n";
    }

    void display() {
        cout << name << " " << roll << endl;
    }

    ~Student() {
        cout << "Destructor called\n";
    }
};

int main() {
    Student *s = new Student("Atiar", 12); // dynamic object

    s->display();   // arrow operator

    delete s;       // free memory

    return 0;
}
```

🔹 Output

```
Constructor called
Atiar 12
Destructor called
```

🔹 Important Points

- `new` returns **pointer**
- `->` used to access members
- `delete` calls **destructor**

✅ 2️⃣ Dynamic Object with User Input

```
Student *s = new Student(name, roll);
```

```
string name;
int roll;

cout << "Enter name: ";
getline(cin, name);

cout << "Enter roll: ";
cin >> roll;

Student *s = new Student(name, roll);
```

✅ 3️⃣ Dynamic Array of Objects

```
int n;
cin >> n;

Student *arr = new Student[n];  // array of objects

delete[] arr;
```

⚠️ Default constructor must exist

✅ 4️⃣ Dynamic Object & Polymorphism (Very Important)

```
class Animal {
public:
    virtual void sound() {
        cout << "Animal sound\n";
    }
    virtual ~Animal() {}
};

class Dog : public Animal {
public:
    void sound() override {
        cout << "Dog barks\n";
    }
};

int main() {
    Animal *a = new Dog();   // polymorphism
    a->sound();

    delete a;
}
```

🔹 Why `delete` is Important?

❌ Without delete → **memory leak**

🔹 `new` vs Stack Object

Stack Object	Heap Object
`Student s;`	`Student *s = new Student();`
Auto destroyed	Manual destroy
Faster	Flexible

🔹 Best Practice (Modern C++)

Use **Smart Pointers**

```
#include <memory>

unique_ptr<Student> s = make_unique<Student>("Atiar", 12);
```

✔ No `delete` needed
✔ Safer

🔹 Interview Q&A

Q: Does `delete` call destructor?
✅ Yes

Q: Can constructor be private?
✅ Yes (factory pattern)

🔹 Built-in Sort Function in C++ (`std::sort`)

C++ provides a **built-in sort function** in the Standard Library called `std::sort()` .
It is **fast**, **easy**, and **widely used** in exams & interviews.

🔹 Header File

```
#include <algorithm>
```

(or `#include <bits/stdc++.h>`)

🔹 Basic Syntax

```
sort(start_iterator, end_iterator);
```

✅ 1️⃣ Sort an Array (Ascending)

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    int arr[] = {5, 2, 9, 1, 3};
    int n = 5;

    sort(arr, arr + n);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}
```

🔹 **Output**

```
1 2 3 5 9
```

✅ 2️⃣ Sort an Array (Descending)

```
sort(arr, arr + n, greater<int>());
```

✅ 3️⃣ Sort a Vector

```
vector<int> v = {4, 1, 7, 2};

sort(v.begin(), v.end());

for (int x : v)
    cout << x << " ";
```

✅ 4️⃣ Sort Using Custom Comparator (Function)

```
bool cmp(int a, int b) {
    return a > b;   // descending
}

sort(v.begin(), v.end(), cmp);
```

✅ 5️⃣ Sort Objects (Class / Struct)

Example: Sort Students by GPA

```
class Student {
public:
    string name;
    double gpa;
};

bool cmp(Student a, Student b) {
    return a.gpa > b.gpa;
}

int main() {
    vector<Student> v = {
        {"Atiar", 3.4},
        {"Rahman", 3.9},
        {"Hasan", 3.2}
    };

    sort(v.begin(), v.end(), cmp);

    for (auto s : v)
        cout << s.name << " " << s.gpa << endl;
}
```

✅ 6️⃣ Sort Using Lambda Function (Modern C++)

```
sort(v.begin(), v.end(), [](int a, int b) {
    return a > b;
});
```

🔹 How `std::sort()` Works?

- Uses **IntroSort**
  - Quick Sort
  - Heap Sort
  - Insertion Sort
- **Time Complexity**:
  - Best / Average / Worst → `O(n log n)`

🔹 Common Mistakes ❌

❌ Forgetting header `<algorithm>`
❌ Wrong range (`arr + n - 1`)
❌ Comparator logic reversed

🔹 Interview Questions

Q: Difference between `sort()` and `stable_sort()`
✅ `stable_sort()` maintains relative order

Q: Can `sort()` be used with linked list?
❌ No (needs random access iterator)

🔹 Quick Summary

Task	Code
Ascending	`sort(a, a+n)`
Descending	`sort(a, a+n, greater<int>())`
Vector	`sort(v.begin(), v.end())`
Custom	`sort(v.begin(), v.end(), cmp)`